Correcciones para R1 — notebook v2

10 puntos. **P0** bloquea la entrega; **P1** se nota en la revisión; **P2** es limpieza.

> **Notación:** `R1 – R4` son los roles del equipo. `P0`, `P1` y `P2` son prioridades, no roles.

P0

1. Quitar `shap` y la explicabilidad del README

Por qué. La justificación de predicciones (SHAP, PDP/ICE, permutation importance, LIME) quedó **excluida** por decisión del equipo. `requirements.txt` instala `shap>=0.45` y el README asigna a R4 una sección "10 Explicabilidad" que no existe en el plan. Un evaluador que lea la carpeta ve una contradicción con el notebook y con lo acordado.

Qué.

- Borrar `shap>=0.45` de `requirements.txt`.
- README, tabla de roles: R4 pasa a **"Conclusiones y documentación"** con secciones **0 Portada, 10 Conclusiones, 11 Referencias** (no 12).
- Cualquier otra mención a SHAP, PDP, LIME o "explicabilidad" fuera de la carpeta.

2. Resolver `categoria` y `product_id`: 25 columnas, 5 duplicadas

Por qué. `categoria` es función de `product_id`: 5 de las 6 categorías tienen un solo producto. Con `drop_first`, sus 5 dummies son **copias exactas** de `product_id_P0006` … `P0010` (la celda 3.6 ya lo imprime). No aporta información y sí distorsiona: en RBF una dimensión duplicada **pesa el doble** en la distancia; en lineal produce colinealidad exacta. `X` tiene 20 columnas de información real, no 25.

Qué. Opción recomendada:

```
CAT = ["product_id"]          # 9 dummies; categoria solo repetía 5 de ellas
```

Y actualizar `assert X.shape == (530, 20)`. Si el equipo decide mantener las dos, hay que escribir en 3.6 y en el informe que la duplicación existe, que duplica el peso del producto en la distancia y que se conserva por continuidad con el plan. "Lo dice el plan" no basta: el plan lo escribimos nosotros.

3. Los 7 warnings de matplotlib 3.11

Por qué. Con las versiones fijadas (`matplotlib 3.11.2` + `seaborn 0.13.2`) los `boxplot` emiten 7 `MatplotlibDeprecationWarning` (`vert=` deprecado). Vienen de `seaborn/categorical.py`, no del notebook, y no se arreglan actualizando seaborn: 0.13.2 es la última versión publicada. La regla del equipo es cero warnings.

Qué. Filtro acotado en la celda 1 (verificado: deja 0 avisos):

```
import warnings
from matplotlib import MatplotlibDeprecationWarning

# seaborn 0.13.2 pasa vert= a matplotlib, deprecado en 3.11.
warnings.filterwarnings("ignore", message=r".*vert.*deprecated.*",
                        category=MatplotlibDeprecationWarning)
```

> No bajar a `matplotlib==3.10.9`: con numpy 2.5 esa versión emite 136 `DeprecationWarning` de `matplotlib/dates.py` y el problema empeora. Con 3.11.2 + filtro son 0.

P1

4. El README cita secciones que no existen

Por qué. Dice "las secciones 3.7 y 3.8 del notebook" y el notebook termina en 3.6. Quien busque esas secciones pierde tiempo y desconfía del resto del documento.

Qué. Cambiar por "3.5 y 3.6".

5. La versión declarada no coincide con la ejecutada

Por qué. El notebook guarda **Python 3.13.18** (pandas 3.0.5, numpy 2.5.3) en su celda de versiones; el README, `.python-version` y el `requirements.txt` de la raíz declaran **3.12.7** con pandas 3.0.6. El entorno de trabajo puede ser el que quieras, pero el notebook y la documentación se copian juntos a la raíz: si dicen cosas distintas, quien reproduzca la práctica con el entorno documentado no obtiene las mismas métricas.

Qué. Ejecutar y guardar con 3.12.7, o cambiar la declaración del repo a la versión que realmente se usó. Lo que no puede quedar es la discrepancia entre el notebook y los tres archivos que dicen la versión.

6. La detección de duplicadas no se entiende

Por qué. `X.columns[X.T.duplicated()]` funciona, pero no se lee, y la salida es una lista sin título: quien revisa no sabe si son columnas duplicadas, excluidas o con nulos.

Qué.

```
duplicadas = X.T[X.T.duplicated()].index.tolist()
display(pd.Series(duplicadas, name="columna idéntica a otra anterior"))
```

Con un Markdown encima que diga que son las dummies de `categoria` y que `P0002` – `P0005` no aparecen porque `categoria_Alimentos` (la de referencia) sería su suma, no una copia.

7. Falta `datos.sha256` en la carpeta

Por qué. El plan pide verificar el dataset con `sha256sum -c datos.sha256` y el archivo no está en `practica-002-v2/`. Sin él no se puede comprobar que los CSV son los mismos que los de la raíz.

Qué. Copiarlo y ejecutar `sha256sum -c datos.sha256`.

P2

8. `historial_original` solo alimenta un `assert`

Por qué. El nombre sugiere que el dataframe se usa para algo más; se crea una variable para una línea y confunde la lectura.

Qué. Inline: `assert pd.read_csv(...).isna().sum().sum() == 0`.

9. La redacción del feriado de enero induce a error

Por qué. La celda 2.5 dice que los feriados caen "al inicio y al final", pero el 1-ene se elimina con `dropna` porque cae en el calentamiento de los rezagos. Tal como está, R2 y R4 pueden concluir que el modelo vio un feriado en entrenamiento, y eso cambia la conclusión.

Qué.

```
Hay tres feriados: el 1-ene, que se pierde con `dropna` porque cae en el calentamiento de los rezagos, y el 28-feb y el 1-mar, que quedan en test. El modelo no ve ningún feriado en entrenamiento (ver 3.6).
```

10. El README de la carpeta no coincide con el README raíz

Por qué. El README que entregas es el que va a quedar en la **raíz del repo**, así que tiene que describir la estructura final y no la carpeta de autor. Hoy dice:

- notebook en `svr_ventas.ipynb`, cuando el plan y el README raíz lo ubican en `src/svr_ventas.ipynb`;
- estructura sin `docs/`, `resultados/modelos/`, `mlflow.db`, `mlartifacts/`, `.python-version`, `datos.sha256`, `requirements-lock.txt` ni `inventory.csv`;
- roles como `P1` – `P4`, cuando el resto del repo ya usa `R1` – `R4`;
- sin mención al Model Registry (`svr_ventas`, una versión por kernel × C, alias `champion`), que es lo que el README raíz documenta;
- comandos escritos para la carpeta de autor, no para la raíz.

Copiado tal cual, el repo queda con dos estructuras descritas y ninguna correcta.

Qué. Tomar como base el README raíz y añadir solo la tabla de estado por rol. En concreto:

- `svr_ventas.ipynb` → `src/svr_ventas.ipynb` en la estructura y en los comandos;
- `P1` – `P4` → `R1` – `R4` en la tabla de estado y en el contrato;
- agregar `docs/`, `resultados/modelos/`, `mlflow.db`, `mlartifacts/`, `.python-version`, `datos.sha256`, `requirements-lock.txt` e `inventory.csv` al árbol;
- describir el registry como en la raíz: `svr_ventas` v1–v16 del barrido, v17 de `GridSearchCV`, alias `champion`;
- los comandos se ejecutan desde la raíz: `MPLBACKEND=Agg jupyter nbconvert --to notebook --execute --inplace src/svr_ventas.ipynb`;
- `requirements.txt`: el de la raíz manda; no sobrescribirlo con el de la carpeta (punto 1 y punto 3). `mlflow` va en `3.18.0`, no `>=2.14`: el registry y `log_model(name=...)` necesitan 3.x.

Cierre

□ `shap` fuera; cero menciones a explicabilidad en la carpeta.

□ README con R4 = Portada, Conclusiones, Referencias (0, 10, 11).

□ `CAT` decidido; `X` con 20 o 25 columnas y `assert` coherente.

□ 0 warnings.

□ Intérprete declarado = intérprete de ejecución.

□ Celda 3.6 legible y con título.

□ `datos.sha256` presente y verificado.

□ Celda 2.5 aclarando el feriado de enero.

□ README con la estructura y las rutas de la raíz (`src/svr_ventas.ipynb`) y con el registry (`svr_ventas`, versiones, alias `champion`).

□ 0 errores y ejecución < 2 min.